The next chapter will continue our discussion of resilience and introduce more patterns, like circuit breakers and rate limiters, using Spring Cloud Gateway, Spring Cloud Circuit Breaker, and Resilience4J.

Summary

- When you expect high traffic and concurrency with fewer computational resources, the reactive paradigm can improve the application's scalability, resilience, and cost-effectiveness at the expense of a steeper initial learning curve.
- Choose between a non-reactive and a reactive stack according to your requirements.
- Spring WebFlux is based on Project Reactor and is the core of the reactive stack in Spring. It supports asynchronous, non-blocking I/O.
- Reactive RESTful services can be implemented through `@RestController` classes or router functions.
- The Spring WebFlux slice can be tested through the `@WebFluxTest` annotation.
- Spring Data R2DBC provides support for reactive data persistence using the R2DBC driver. The approach is the same as for any Spring Data project: database drivers, entities, and repositories.
- Database schemas can be managed with Flyway.
- The persistence slice of a reactive application can be tested using the `@DataR2dbcTest` annotation and Testcontainers.